

FEM–OSR Fatigue Pipeline

User Manual — Version 1.0

Lukas Deplace

Division of Solid Mechanics, Lund University

August 24, 2026

This manual documents the FEM–OSR pipeline developed at the Division of Solid Mechanics, Lund University, during a 4-month engineering internship. The pipeline couples Ansys Mechanical to a Fortran implementation of the Ottosen–Stenström–Ristinmaa (OSR) continuum high-cycle fatigue model, and provides an automated workflow inside Workbench for arbitrary geometries and loading configurations.

Contents

1	Overview	3
1.1	What the pipeline does	3
1.2	Architecture at a glance	3
2	Installation and setup	3
2.1	Setup video 1 — Folder structure and Fortran build	3
2.2	Setup video 2 — Configuring the shared folder	4
2.3	Setup video 3 — Installing the Workbench extension	5
2.4	Prerequisites	6
3	Tutorials	6
3.1	Tutorial overview	6
3.2	Square benchmark — External Data setup	7
3.3	Square benchmark — Workbench setup, first run and damage import	7
3.4	Perforated plate — Two load cases and Super Solve	8
3.5	Perforated plate — Result and damage visualisation	9
3.6	Test OSR only — Changing the phase shift	9
3.7	Test OSR only — Updated damage result	10
3.8	Additional examples	10
4	File formats reference	10
4.1	loadcases.txt	10
4.2	Stress field format	11
4.3	History file format	11
4.4	Output files	11
4.5	APDL export macro	11
5	Advanced configuration	12
5.1	OSR material parameters	12
5.2	Choosing the integrator	13
5.3	Adjusting DOPRI tolerances	13
5.4	OpenMP parallelisation	13
5.5	Table of hard-coded parameters	13
6	Troubleshooting	13
7	Possible extensions	14
7.1	Adding a new integration scheme	14
7.2	Adding a new loading mode	14
8	Known limits and perspectives	14

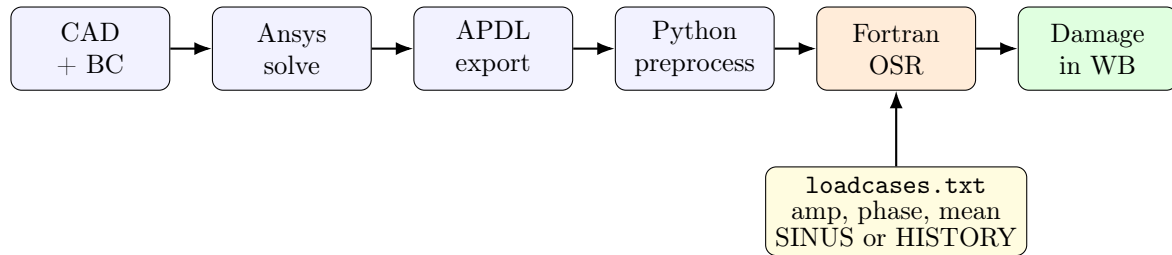
1 Overview

1.1 What the pipeline does

The pipeline predicts high-cycle fatigue damage in 2D or 3D structures under multiaxial, non-proportional, and possibly non-periodic loadings. It takes as input the elastic stress fields computed by Ansys for a set of unit load cases, combines them by tensor superposition, and integrates the OSR damage equations at every point of the mesh in parallel.

The output is a damage field $D \in [0, 1]$ and a number of cycles to failure $N_{\min}$, re-imported into Workbench for visualisation.

1.2 Architecture at a glance



2 Installation and setup

The installation is documented in three short videos. Each video is about 2–3 minutes long and walks through the setup on a fresh Windows machine. The text below summarises the essential points; refer to the videos for the full step-by-step.

The handover archive already follows the working directory layout used in the tutorial videos. In the first setup video, the folders are created manually to explain the role of each directory. When using the archive, these folders are already provided; the important point is to keep the same relative structure.

2.1 Setup video 1 — Folder structure and Fortran build

Companion video: Setup 1 — Folder structure and Fortran build

File to open from the accompanying tutorials/ folder:
tutorials/setup/setup_1_folders_and_build.mp4

This first video presents the reference directory layout and compiles the Fortran solver.

Starting from the main directory, called `OSR_tutorial` in the video but arbitrary in practice, the reference structure is:

```

1 OSR_tutorial/
2 |-- OSR_model/           Fortran source code + Makefile + Python driver
3 |-- OSR_cases/           Ansys projects or test-case workspaces
4 |-- shared/              Exchange folder between Ansys and the pipeline
5 |-- ansys_extension/     Workbench extension files
6 |-- tutorials/           Companion tutorial videos
7 |   |-- setup/
8 |   |-- test_cases/
  
```

The `OSR_model/` folder must contain the Fortran source files (`osr_model.f90`, `osr_analytical.f90`, `pipeline_aux.f90`, `osr_pipeline_multi.f90`), the `Run_pipeline.py` script, and the `Makefile`. Then compile:

```
1 cd OSR_model
2 make
```

The resulting `osr_pipeline_multi.exe` is the executable launched by the Python driver.

Note

The Makefile assumes `gfortran` is in the `PATH` and has OpenMP support. On Windows, use MSYS2 or a MinGW-w64 distribution.

2.2 Setup video 2 — Configuring the shared folder

Companion video: Setup 2 — Shared folder configuration

File to open from the accompanying `tutorials/` folder:
`tutorials/setup/setup_2_shared_folder.mp4`

The `shared/` folder is the exchange point between Ansys, the Python driver, and the Fortran solver. In the handover archive, this folder is already provided with the initial files needed for a first setup. The second setup video explains the role of this folder and how to adapt its configuration files on a new machine.

It contains the following initial files:

File	Role
<code>loadcases.txt</code>	Loading configuration (see Section 4.1)
<code>act_config.json</code>	Paths and execution settings for the Workbench extension and Python driver
<code>history.txt</code>	Example tabulated loading history (HISTORY mode only)
<code>APDL.txt</code>	APDL snippet to paste into an Ansys Commands block
<code>damage.csv</code>	Placeholder damage file used to configure External Data before the first run

The main configuration to adapt on your machine is `act_config.json`:

```
1 {
2   "python_exe":      "C:/Python39/python.exe",
3   "pipeline_script": "C:/Users/.../OSR_tutorial/OSR_model/
  Run_pipeline.py",
4   "shared_dir":      "C:/Users/.../OSR_tutorial/shared",
5   "omp_threads":     8
6 }
```

The key names must not be changed: the Workbench extension expects `python_exe`, `pipeline_script`, `shared_dir`, and `omp_threads`. Only the paths and the number of threads should be adapted to the local machine.

Loadcases file at a glance. Before showing the mode switch, here is what `loadcases.txt` looks like. The full reference is in Section 4.1.

```
1 # n_cases  omega  mode
2 1  6.2831853  SINUS
3
4 # file  amp  phase  mean
```

```

5 sigma_export_nodes_tutorial.txt 1.0 0.0 0.0
6
7 # n_cycles_max n_steps_per_cycle n_substeps_history
8 200000 100 20
9
10 # history_file (dash "-" if not HISTORY mode)
11 -
12
13 # integrator (RK4, DOPRI, DOPRI_EVENTS, AUTO)
14 AUTO

```

Two loading modes are supported:

- **SINUS** — periodic sinusoidal loading. The amplitude, phase, and mean are read directly from `loadcases.txt`. The `history_file` field is set to `-`.
- **HISTORY** — arbitrary tabulated loading. The $\lambda_k(t)$ values come from an external file (`history.txt`), and the amp/phase/mean columns are ignored.

The video demonstrates switching between the two modes by editing this file.

Warning

The order of the columns in `history.txt` must match the order of the stress files listed in `loadcases.txt`. Otherwise the pipeline will superpose the wrong stress fields with the wrong loading multipliers.

2.3 Setup video 3 — Installing the Workbench extension

Companion video: Setup 3 — Workbench extension

File to open from the accompanying `tutorials/` folder:
`tutorials/setup/setup_3_workbench_extension.mp4`

Warning

The setup video shows how to install the Workbench extension. However, when using the handover archive on a new machine, one additional check is required before loading the extension.

Open the IronPython action script:

```
1 ansys_extension/SuperSolve/super_solve_action.py
```

At the beginning of this file, update the two local paths:

```
1 CONFIG_FILE = r"C:\path\to\OSR_tutorial\shared\act_config.json"
2 DEFAULT_SHARED_DIR = r"C:\path\to\OSR_tutorial\shared"
```

The first path tells the extension where to find `act_config.json`. The second path is used to write the ACT debug and error logs. Both paths must point to the local `shared/` folder of the installation.

The **SuperSolve** extension adds three buttons to the Workbench toolbar (**Super Solve**, **Test OSR only**, **Refresh damage**).

Inside the `ansys_extension/` folder, place the `SuperSolve/` subfolder (which contains `super_solve_action.py`) and the `SuperSolve.xml` definition file.

In Workbench:

1. *Extensions* → *Install Extension* → select **SuperSolve.xml**
2. *Extensions* → *Manage Extensions* → tick **SuperSolve**

The three OSR buttons then appear in the toolbar.

Tip

If the extension does not show up in *Manage Extensions*, close and restart Workbench, then reopen *Manage Extensions*.

2.4 Prerequisites

Component	Version
Ansys Mechanical	2025 R1 or later (needs Workbench + ACT support)
Fortran compiler	gfortran ≥ 11 , with OpenMP support
Python	≥ 3.9
Python packages	numpy
Operating system	Windows 10/11 or Linux (tested on both)

3 Tutorials

This section follows the exact video sequence used in the tutorial series. The objective is to first verify the complete workflow on a simple square benchmark, then to move to the perforated plate with two Ansys load cases, and finally to show how **Test OSR only** can be used to change OSR-side loading parameters without re-solving the finite element model.

The initial input files are delivered directly in the **shared/** folder of the handover archive. The videos are intentionally short: some computations are cut in the recording, while the log files are used to show what happened during the pipeline run.

Note

The **shared/** folder contains a placeholder **damage.csv** with the correct header format (**X,Y,Z,Damage** and a few dummy rows). This file is needed to configure the External Data block **before** the first pipeline run. After the first successful Super Solve, the pipeline overwrites the same file with the real damage field.

3.1 Tutorial overview

Video	Purpose
Square 1	Configure the External Data block used to read damage.csv from the shared/ folder.
Square 2	Configure the square benchmark in Workbench, insert the Commands objects, launch Super Solve and refresh and import the damage field after the square computation

Plate 1	Configure the perforated plate with two Static Structural load cases and launch Super Solve .
Plate 2	Inspect <code>pipeline_log.txt</code> and import the resulting damage field.
Plate 3	Modify the phase shift in <code>loadcases.txt</code> and launch Test OSR only .
Plate 4	Inspect the log and reload the updated damage field after the OSR-only run.

3.2 Square benchmark — External Data setup

Companion video: External Data setup — Reading damage.csv

File to open from the accompanying `tutorials/` folder:
`tutorials/test_cases/tutorial_external_data_setup.mp4`

This first tutorial configures the External Data block used to read the OSR damage file back into Workbench. This step must be done **before** the first Super Solve, because the pipeline needs an existing External Data connection to display the result.

Open the *Setup* cell of the External Data system and select the placeholder `damage.csv` from the `shared/` folder. This file is only a dummy file used to configure the External Data block before the first pipeline run. After the first successful **Super Solve**, the pipeline overwrites the same `damage.csv` file with the real damage field.

In the column definition table, assign the first three data lines to the spatial coordinates:

- X Coordinate;
- Y Coordinate;
- Z Coordinate.

For a 2D case, the Z coordinate is kept equal to zero. The last data line is selected as *Temperature* and its data identifier is renamed `damage`. Since the first line of `damage.csv` is a header `X,Y,Z,Damage`, set **Start Import At Line** to 2. For the square benchmark, set the dimension to 2D and the length unit to millimetres.

Warning

The damage field is imported as a *Body Temperature* load in Mechanical. This is only a visualisation trick: the displayed values correspond to the dimensionless OSR damage variable, not to a physical temperature.

After a pipeline run, refreshing this External Data block reloads the latest `damage.csv` file and makes the damage field available in Mechanical.

3.3 Square benchmark — Workbench setup, first run and damage import

Companion video: Square benchmark — Workbench setup, Super Solve and damage import

File to open from the accompanying `tutorials/` folder:
`tutorials/test_cases/tutorial_square_super_solve.mp4`

Now that the External Data block is configured, this tutorial sets up the square benchmark itself. Its role is to check the complete connection between Ansys stress export, the OSR solver, and damage visualisation in Mechanical.

The Workbench project contains a Static Structural system and an External Data system used for the damage field. In Mechanical, insert a first *Commands* object under *Static Structural* containing only:

```
1 ERESX, NO
```

This keeps a peak-preserving stress representation for the fatigue calculation. Then insert a second *Commands* object under the *Solution* branch. Paste the APDL extraction script from `APDL.txt`. In this block, check two lines carefully:

- the element type selection line, for example `ESEL,S,ENAME,,183` for `PLANE183`;
- the output filename in the `*CFOPEN` line, which must match the stress file declared in `loadcases.txt`.

Finally, return to Workbench and click **Super Solve**. The button updates the Ansys model, exports the stress field, runs the Python preprocessing, launches the Fortran OSR solver, and writes `damage.csv` (overwriting the placeholder). For this simple square benchmark, the computation is short enough to be followed directly.

Once the computation is finished, click **Refresh Damage** in Workbench. When Workbench asks whether the upstream data should be re-read, click *Yes*. Then open Mechanical, right-click the imported *Body Temperature* field, and select *Import Load*.

For the simple square benchmark, the damage field is uniform. Therefore, no visible colour gradient is expected. The important check is that the imported minimum and maximum values are both equal to the expected damage level.

Tip

From this point on, any subsequent run follows a simpler workflow: edit `loadcases.txt`, click **Test OSR only** if the stress files are already available, then **Refresh Damage**.

3.4 Perforated plate — Two load cases and Super Solve

Companion video: Perforated plate — Two load cases and Super Solve

File to open from the accompanying `tutorials/` folder:

`tutorials/test_cases/tutorial_plate_two_loadcases_super_solve.mp4`

This tutorial moves from the square benchmark to a perforated plate with two separate loading directions.

The Workbench project contains three systems:

- `holed_plank_horizontal`, a Static Structural system for the horizontal load case;
- `holed_plank_vertical`, a Static Structural system for the vertical load case;
- `visualisation`, a final system used only to display the damage field. The External Data system is connected to this visualisation system.

In the horizontal load case, a pressure of -200 MPa is applied on the left edge in the horizontal direction. The Commands objects are already inserted as in the previous tutorial, but the APDL output filename must be checked. In the Commands object under the Solution branch, the exported stress file is named:

```
1 sigma_export_nodes_tutorial_horizontal.txt
```

In the vertical load case, a pressure of -200 MPa is applied on the top edge in the vertical direction. The second APDL export must write a different stress file:

```
1 sigma_export_nodes_tutorial_vertical.txt
```

Then open `loadcases.txt`. The initial file contains only one stress file, for example:

```
1 1 6.283185307 SINUS
2 sigma_export_nodes_tutorial.txt 1.0 0.0 0.0
```

Replace this single entry by the vertical file, duplicate the line for the horizontal file, and change `n_cases` from 1 to 2:

```
1 # n_cases  omega  mode
2 2 6.283185307  SINUS
3
4 # file  amp  phase  mean
5 sigma_export_nodes_tutorial_vertical.txt      1.0  0.0  0.0
6 sigma_export_nodes_tutorial_horizontal.txt    1.0  0.0  0.0
7
8 # n_cycles_max  n_steps_per_cycle  n_substeps_history
9 200000  100  1
10
11 # history_file
12 -
13
14 # integrator
15 RK4
```

The order of the two stress files is important, because it defines the order in which the OSR solver combines the static stress tensors. After saving the file, return to Workbench and click **Super Solve**. During the computation, the pipeline updates both Ansys load cases, exports both stress fields, preprocesses them, and reconstructs the time-dependent multiaxial stress tensor by superposition inside the OSR solver.

3.5 Perforated plate — Result and damage visualisation

Companion video: Perforated plate — Result and damage visualisation

File to open from the accompanying `tutorials/` folder:
`tutorials/test_cases/tutorial_plate_result_refresh_damage.mp4`

This video starts after the two-loadcase computation has finished. First, open `pipeline_log.txt` in the shared folder and highlight the lines corresponding to the last simulation. This file records the main execution steps: stress-file processing, Fortran OSR computation, generated output files, and the final pipeline status.

Then return to Workbench and click **Refresh Damage**. As usual, open the visualisation setup, accept the upstream data reload message, and go back into Mechanical. Right-click the imported *Body Temperature* field and select *Import Load*.

The OSR damage field is now displayed on the perforated plate. Unlike the square benchmark, the damage is non-uniform and the highest values appear around the stress concentration near the hole.

3.6 Test OSR only — Changing the phase shift

Companion video: Test OSR only — Changing the phase shift

File to open from the accompanying `tutorials/` folder:
`tutorials/test_cases/tutorial_run_osr_only_phase_shift.mp4`

This tutorial uses the same biaxial perforated-plate project, but changes only the loading definition. The Ansys stress fields have already been exported, so there is no need to re-run the Static Structural systems.

Open `loadcases.txt` and change the phase of the second load case from 0.0 to 1.5707963268, i.e. $\pi/2$:

```

1 # file amp phase mean
2 sigma_export_nodes_tutorial_vertical.txt 1.0 0.0 0.0
3 sigma_export_nodes_tutorial_horizontal.txt 1.0 1.5707963268 0.0

```

This introduces a 90-degree phase shift between the two loading directions. The finite element stress fields remain unchanged; only their time-dependent combination changes. After saving `loadcases.txt`, return to Workbench and click **Test OSR only**.

This button skips the Ansys mechanical solves and reuses the already exported stress files. It runs only the Python preprocessing and the Fortran OSR solver, which makes it useful for testing different amplitudes, phase shifts, mean values, or HISTORY signals.

3.7 Test OSR only — Updated damage result

Companion video: Test OSR only — Updated damage result

File to open from the accompanying `tutorials/` folder:
`tutorials/test_cases/tutorial_run_osr_only_result.mp4`

After the OSR-only computation has finished, open `pipeline_log.txt` to check the execution and confirm that the OSR run completed successfully. Then return to Workbench, click **Refresh Damage**, open the visualisation setup, accept the upstream data reload message, and import the *Body Temperature* load again in Mechanical.

The displayed damage field now corresponds to the new phase-shifted loading configuration. Since only the phase was changed, the result illustrates the main advantage of the superposition architecture: different loading paths can be explored without re-solving the finite element model.

3.8 Additional examples

The same workflow can be extended to more advanced cases, such as a 3D bracket with clamp filtering or arbitrary non-periodic HISTORY loading. These examples use the same basic sequence: prepare one Ansys stress export per unit load case, list the corresponding files in `loadcases.txt`, run either **Super Solve** for a full update or **Test OSR only** when only the loading definition has changed, and refresh the damage field through External Data.

For HISTORY mode, the loading multipliers are read from `history.txt`. The first column is time and the following columns are the load multipliers in the same order as the stress files in `loadcases.txt`. The recommended integrator is `DOPRI_EVENTS` for irregular tabulated signals.

4 File formats reference

4.1 loadcases.txt

The loading configuration is a plain-text file read by both the Python driver and the Fortran solver. Lines starting with `#` and blank lines are ignored.

Line	Content
1	<code>n_cases omega mode</code>
2 to $n_{\text{cases}} + 1$	For each unit case: <code>filename amp phase mean</code>

Next	n_cycles_max n_steps_per_cycle
	n_substeps_history
Next	history_file (name of the history file, or - in SINUS mode)
Last	integrator (one of RK4, DOPRI, DOPRI_EVENTS, AUTO)

4.2 Stress field format

Each `sigma_export_nodes_*.txt` file has 11 space-separated columns:

```
elem_id  node_id  X  Y  Z  SXX  SYX  SZZ  SXY  SYZ  SXZ
```

One line per (element, corner-node) pair after ERESX,NO extraction. Comment lines start with #.

Binary cache files (.bin) hold the same data as raw float64 values, one 88-byte record per point, in native little-endian order.

4.3 History file format

For a run with K load cases:

```
t  lambda1  lambda2  ...  lambdaK
```

Sample times must be strictly increasing. Interpolation is piecewise linear between samples.

4.4 Output files

File	Content
damage.csv	CSV for Workbench External Data: X,Y,Z,Damage
damage_field.txt	Full-detail ASCII: elem_id node_id X Y Z D N_fail
critical_points.txt	Top-20 most damaged points, sorted by N_{fail}
pipeline_log.txt	Standard output of the Fortran run

4.5 APDL export macro

The following macro extracts the stress tensor at every corner node of every element and writes it to a text file compatible with the pipeline. Adjust the element type (187 for SOLID187 in 3D, 183 for PLANE183 in 2D) and the output filename in *CFOPEN.

```

1 /POST1
2 SET, LAST
3 RSYS, 0
4 AVRES, , NOAV
5 ERESX, NO
6
7 ESEL, S, ENAME, , 187      ! or 183 for 2D
8 *GET, nb_elem, ELEM, , COUN
9
10 *DIM, EID_LIST, ARRAY, nb_elem
11 *GET, e_tmp, ELEM, , NUM, MIN
12 EID_LIST(1) = e_tmp
13 *DO, ie, 2, nb_elem
14     *GET, e_tmp, ELEM, e_tmp, NXTH
15     EID_LIST(ie) = e_tmp

```

```

16 *ENDDO
17
18 nb_pts = nb_elem * 4
19 *DIM, A_EID, ARRAY, nb_pts
20 *DIM, A_NID, ARRAY, nb_pts
21 *DIM, A_CX, ARRAY, nb_pts
22 *DIM, A_CY, ARRAY, nb_pts
23 *DIM, A_CZ, ARRAY, nb_pts
24 *DIM, A_SX, ARRAY, nb_pts
25 *DIM, A_SY, ARRAY, nb_pts
26 *DIM, A_SZ, ARRAY, nb_pts
27 *DIM, A_SXY, ARRAY, nb_pts
28 *DIM, A_SYZ, ARRAY, nb_pts
29 *DIM, A_SXZ, ARRAY, nb_pts
30
31 *DO, ie, 1, nb_elem
32     eid_cur = EID_LIST(ie)
33     ESEL, S, ELEM, , eid_cur
34     *DO, k, 1, 4
35         *GET, nid, ELEM, eid_cur, NODE, k
36         idx = (ie-1)*4 + k
37         A_EID(idx) = eid_cur
38         A_NID(idx) = nid
39         *GET, A_CX(idx), NODE, nid, LOC, X
40         *GET, A_CY(idx), NODE, nid, LOC, Y
41         *GET, A_CZ(idx), NODE, nid, LOC, Z
42         *GET, A_SX(idx), NODE, nid, S, X
43         *GET, A_SY(idx), NODE, nid, S, Y
44         *GET, A_SZ(idx), NODE, nid, S, Z
45         *GET, A_SXY(idx), NODE, nid, S, XY
46         *GET, A_SYZ(idx), NODE, nid, S, YZ
47         *GET, A_SXZ(idx), NODE, nid, S, XZ
48     *ENDDO
49 *ENDDO
50
51 ALLSEL, ALL
52 *CFOPEN, 'C:\path\to\shared\sigma_export_nodes_horizontal', 'txt'
53 *VLEN, nb_pts
54 *VWRITE, A_EID(1), A_NID(1), A_CX(1), A_CY(1), A_CZ(1), A_SX(1), A_SY
55         (1), A_SZ(1), A_SXY(1), A_SYZ(1), A_SXZ(1)
56 (F8.0, F8.0, 3E14.6, 6E14.6)
57 *CFCLOS

```

5 Advanced configuration

5.1 OSR material parameters

The OSR model has five material parameters, set at the top of `osr_pipeline_multi.f90`:

```

1 p%A      = 0.225_dp
2 p%sigma_oe = 490.0_dp
3 p%C      = 1.25_dp
4 p%K      = 2.65e-5_dp
5 p%L      = 14.4_dp

```

The values above are calibrated for AISI-SAE 4340 steel by Ottosen *et al.* (2008). To use another material, replace them with your calibrated values (Haigh diagram for A and σ_{oe} , then least-squares fit on the experimental Wöhler curve for C , K , L).

5.2 Choosing the integrator

Integrator	When to use
RK4	Fixed-step, order 4. Recommended for SINUS mode with periodic signals. Predictable CPU cost.
DOPRI	Dormand–Prince RK5(4)7M, adaptive. Good for smooth HISTORY signals.
DOPRI_EVENTS	Same as DOPRI, plus event detection at $\beta = 0$. Recommended for irregular HISTORY signals with frequent activation changes.
AUTO	Default: RK4 in SINUS, DOPRI_EVENTS in HISTORY. Suitable for most use cases.

5.3 Adjusting DOPRI tolerances

If DOPRI gives too much or too little precision for your use case, edit the constants in `osr_model.f90` inside `osr_integrate_dopri`:

```
1 real(dp), parameter :: ATOL = 1.0e-6_dp
2 real(dp), parameter :: RTOL = 1.0e-4_dp
```

Tighter values (1e-8 / 1e-6) give more precision at higher CPU cost. Recompile after any change.

5.4 OpenMP parallelisation

The number of threads is controlled by the `OMP_NUM_THREADS` environment variable set by `Run_pipeline.py` from `omp_threads` in `act_config.json`. Set it to the number of physical cores on your machine.

5.5 Table of hard-coded parameters

For advanced tuning, here are the tunable constants scattered across the source files:

Parameter	Meaning	Default	Location
NSAMP_BETA	Pre-filter samples per cycle	128	<code>Run_pipeline.py</code>
CLAMP_RADIUS	Exclusion radius around clamps	15.0 mm	<code>Run_pipeline.py</code>
ATOL / RTOL	DOPRI tolerances	1e-6 / 1e-4	<code>osr_model.f90</code>
SAFETY	DOPRI step controller safety	0.9	<code>osr_model.f90</code>
FACMIN / FACMAX	Step growth/shrink bounds	0.1 / 5.0	<code>osr_model.f90</code>
MAX_STEPS	Safety cap on DOPRI iterations	1e7	<code>osr_model.f90</code>
USE_CLAMP_FILTER	Enable or disable clamp singularity filtering	False	<code>Run_pipeline.py</code>
USE_DEDUPLICATION	Enable or disable max- β deduplication	True	<code>Run_pipeline.py</code>

6 Troubleshooting

Symptom	Diagnosis and fix
[tl] $N_{\min}$ ridiculously low (e.g. 10^{-9})	Clamp singularities may not be filtered. Set <code>USE_CLAMP_FILTER = True</code> in <code>Run_pipeline.py</code> , and populate <code>CLAMP_CENTERS</code> with the coordinates of the bolt-hole centres.
OpenMP results not reproducible	A shared variable that should be private. Check that all thread-local state (<code>sigma_at_point</code>) is declared <code>THREADPRIVATE</code> and that no line accidentally rewrote a shared variable inside the OpenMP loop.
Binary cache seems stale	<code>binary_cache_valid</code> compares timestamps. If a <code>loadcases.txt</code> or <code>history.txt</code> was modified, the cache is invalidated. To force reuse: touch the <code>.bin</code> files.
Deduplication takes very long in HISTORY	Every sample of the history is used to evaluate β . Sub-sample the history (change the loop in <code>Run_pipeline.py</code> to stride through every N -th sample).
DOPRI takes 0 accepted steps	Step size too large. Increase <code>n_initial_steps</code> in the caller, or reduce <code>hmax</code> in <code>osr_integrate_dopri</code> .
Fortran stop 1 on binary file size not multiple of 88	Corrupted binary cache. Delete the <code>.bin</code> file and let the driver regenerate it from the <code>.txt</code> .
[tl]Ansys reports a stress file access error	Check that the folder in <code>*CFOPEN</code> exists and is writable, and that no antivirus is quarantining the file.

7 Possible extensions

This section is not required for running the pipeline. It only indicates the main extension points for a future developer who wants to modify the code.

7.1 Adding a new integration scheme

New integration schemes should follow the same callback interface as the existing OSR integrators. After implementing the new routine, register it in the integrator selector of `osr_pipeline_multi.f90`, where the value read from `loadcases.txt` is mapped to the corresponding integration routine.

7.2 Adding a new loading mode

New loading modes should follow the structure of `MODE_SINUS` and `MODE_HISTORY` in the pipeline state module. This requires adding a new mode identifier, updating the load-case parser, and defining the functions used to reconstruct $\lambda_k(t)$ and its derivative.

8 Known limits and perspectives

Numerical limitations and remaining perspectives (pipeline-specific)

- DOPRI event detection uses a reject-and-halve approach rather than exact event localisation by re-integrating exactly up to the event. This strategy is robust and simple, but for highly irregular signals with many $\beta = 0$ crossings, the event location may still be less accurate than a fully event-driven integration scheme.
- The HISTORY interval lookup was initially performed by a linear scan of the tabulated time grid. This became a significant bottleneck for long histories. It has now been replaced by a binary-search procedure, which reduced the average integration time from approximately 40.29 s to 4.22 s on the tested HISTORY benchmark, corresponding to a speed-up of about $9.6\times$. The remaining cost for very long HISTORY signals is therefore dominated mainly by the number of time intervals and OSR evaluations, rather than by the interval-search procedure itself.
- The stress extraction and preprocessing strategy remains dependent on the finite element representation. The current workflow uses element-wise nodal stresses exported with `ERESX,NO` in order to preserve local peaks, and optional filtering is used to remove clamp-related singularities. As a result, very local stress concentrations should still be interpreted with care, especially when comparing different meshes or geometries.